

FixTime — Plataforma de Agendamento

Relatório Consolidado de Publicação, Refinamento por Testes, Governança e Análise Ético-Técnica de IA

Repositório Público: github.com/Fixtime-Bootcamp/bootcamp3

Release Branch: main

Status dos Testes: 104 TESTES • 100% PASS

1 Identificação dos Integrantes da Equipe

Caio Boudens de Castro

RA: 22304522

Eduardo Frois Drumond

RA: 22303035

Fernando Medeiros Farias

RA: 22306100

Larissa Queiroz Ramos

RA: 22304308

Mayssa Barbosa Dias

RA: 22303603

Thiago Venâncio Gomides

RA: 22307398

2 Arquitetura da Solução e Fluxo Operacional

O **FixTime** é uma aplicação completa e determinística projetada para eliminar conflitos de agendamento em pequenas e médias assistências técnicas. O sistema é baseado no modelo **Spec-Driven Development (SDD)**, onde a especificação formal governa 100% da implementação e dos contratos da API REST.

Frontend SPA

React 19, TypeScript, Vite. Telas de agenda, cadastros de clientes/técnicos/serviços, ações de status, paginação e exportação CSV.

Backend API

Java 21, Spring Boot 3.4.5. Regras de negócio restritas aos Services: antecedência mínima (2h), jornada útil, feriados nacionais e conflitos.

Persistência & Docker

PostgreSQL em produção/Compose e H2 Database em testes. Docker Compose com inicialização via comando único.

3 Governança de Código e Evidências de Revisão por Pares

A colaboração da equipe seguiu estritamente o fluxo de **Feature Branches** originadas a partir da branch **develop**. Todo código foi integrado exclusivamente via **Pull Requests detalhados** com code review entre pares, critérios formais de aceitação cumpridos e exclusão automática de branches pós-merge:

PR	Título / Funcionalidade Integrada	Módulo	Status	Link GitHub
#24	Definir contrato definitivo de agendamento e SDD	Fundação / SDD	MERGED	Ver PR #24
#25	Padronizar tratamento e respostas de erros HTTP (GlobalExceptionHandler)	Backend / API	MERGED	Ver PR #25
#29	Implementar entidade JPA e repository de agendamentos	Backend / JPA	MERGED	Ver PR #29
#31	Expandir test harness com testes unitários e de integração	Qualidade / Testes	MERGED	Ver PR #31
#33	Tornar docker compose e healthchecks reproduzíveis	DevOps / Docker	MERGED	Ver PR #33
#40	Integrar cliente tipado da API no Frontend	Frontend / Client	MERGED	Ver PR #40
#48	Implementar fluxo de criação e modal de agendamento	Frontend / UI	MERGED	Ver PR #48
#50	Implementar agenda operacional e ações de status	Frontend / UI	MERGED	Ver PR #50
#51	Telas de cadastro e gestão de Clientes, Técnicos e Serviços	Fullstack	MERGED	Ver PR #51
#52	Bloqueio de feriados nacionais e indisponibilidades na agenda	Backend / Regras	MERGED	Ver PR #52
#58	Exportação de agendamentos em CSV com streaming UTF-8 BOM (RFC 4180)	Fullstack / Export	MERGED	Ver PR #58
#74	Documentação interativa da API com OpenAPI / Swagger UI	Docs / Swagger	MERGED	Ver PR #74
#77	Diagramas de fluxo, relatório de erros lógicos e análise ética de IA	Governança / Docs	MERGED	Ver PR #77

4 Relatório de Execução dos Testes Automatizados (100% Pass)

104

TESTES
AUTOMATIZADOS

44

BACKEND (JUNIT 5)

60

FRONTEND (VITEST)

100%

TAXA DE SUCESSO
(ZERO FALHAS)

LOG DE EXECUÇÃO: BACKEND (mvn clean test)

BUILD SUCCESS (44/44 PASS)

```
Running com.fixtime.appointment.AppointmentCsvWriterTest (3 tests) - OK Running
com.fixtime.AppointmentServiceTest (17 tests) - OK (Regras RN01-RN08 cobertas) Running
com.fixtime.AppointmentIntegrationTest (11 tests) - OK (MockMvc + H2) Running
com.fixtime.AppointmentExportIntegrationTest (4 tests) - OK (RFC 4180 / UTF-8 BOM) Running
com.fixtime.BlockedDateIntegrationTest (5 tests) - OK (Feriados Nacionais Fixos) Running
com.fixtime.CustomerServiceTest (3 tests) - OK Running com.fixtime.HealthControllerTest (1 test) - OK
[INFO] Tests run: 44, Failures: 0, Errors: 0, Skipped: 0 [INFO] -----
----- [INFO] BUILD SUCCESS - Total time: 15.341 s
```

LOG DE EXECUÇÃO: FRONTEND (vitest --run)

ALL SUITES PASSED (60/60 PASS)

```
✓ src/utils/appointmentEligibility.test.ts (7 tests) ✓ src/api/client.test.ts (3 tests) ✓
src/components/AppointmentCard.test.tsx (7 tests) ✓ src/pages/TechniciansPage.test.tsx (7 tests) ✓
src/pages/CustomersPage.test.tsx (10 tests) ✓ src/pages/ServicesPage.test.tsx (8 tests) ✓
src/components/CreateAppointmentModal.test.tsx (8 tests) ✓ src/App.test.tsx (10 tests) Test Files 8
passed (8) Tests 60 passed (60) Duration 19.69s (100% Green)
```

5 Inspeção de Erros Lógicos e Loop de Re-especificação (SDD)

O ciclo **Spec-Driven Development** garantiu que nenhuma falha lógica fosse corrigida de forma empírica. Sempre que um erro sutil era revelado pelos testes automatizados, a especificação técnica formal (docs/SPEC.md) e o histórico de refinamentos (docs/spec-changelog.md) eram atualizados antes da refatoração:

- **1. Duração Arbitrária de Serviços (Issues #1 e #8):** O payload inicial permitia que o usuário enviasse `durationMinutes` arbitrário. Corrigido com vinculação estrita à entidade `ServiceEntity` e cálculo determinístico no backend: `endsAt = startsAt + service.duration`.
- **2. Bloqueio Indevido de Horários Adjacentes (RN05 / Issue #10):** Comparações SQL inclusivas bloqueavam visitas contíguas (ex.: 09:00-10:00 e 10:00-11:00). Corrigido com intervalos abertos nas extremidades: `startsAt < other.endsAt AND endsAt > other.startsAt`.
- **3. Flutuação Temporal e Testes Flaky (RN03 / Issue #17):** O uso de `Instant.now()` causava falhas aleatórias por frações de milissegundo na validação de 2h de antecedência. Solucionado com injeção de dependência de `java.time.Clock` imutável nos testes e nos services.
- **4. Tentativa de Conclusão Prematura (RN07 / Issue #13):** O sistema permitia marcar visitas como concluídas antes da realização. Bloqueado com validação estrita exigindo `clock.instant().isAfter(appointment.getEndsAt())`.
- **5. Mojibake no CSV (RF08 / Issue #28):** Exportações de clientes e serviços acentuados abriam corrompidos no Excel. Corrigido com inclusão de streaming com UTF-8 BOM (0xEF, 0xBB, 0xBF) e cabeçalhos RFC 4180.

6 Análise Comparativa e Avaliação Crítica de IA (Item 4 Obrigatório)

6.1. Matriz Comparativa entre Ferramentas de IA Avaliadas

Critério de Avaliação	Claude Code (CLI)	OpenAI Codex / CLI	Cursor (IDE)	Antigravity (IDE/Agente)
Interface de Operação	Terminal / CLI	Terminal / Scripts	IDE (VS Code Fork)	IDE / Agente Autônomo
Compreensão do Repositório	Alta (Busca ativa)	Média (Janela limitada)	Muito Alta (Embeddings)	Muito Alta (Knowledge Items)
Aderência ao Modelo SDD	Alta (Raciocínio lógico)	Média (Foco em boilerplate)	Alta (via @docs/SPEC.md)	Excelente (Regras nativas)
Geração de Testes Automatizados	Excelente (Edge cases)	Boa (Caminho feliz)	Muito Boa (Mocks rápidos)	Excelente (Docstrings/AssertJ)
Execução Autônoma de Comandos	Alta (Shell interativo)	Baixa (Requer orquestração)	Média (Terminal assistido)	Total (Builds, tasks e logs)
Propensão a Alucinação	Baixa	Média	Baixa a Média	Muito Baixa (Auto-inspeção)
Governança de Branches / Regras	Alta	Baixa	Alta (via .cursorsrules)	Muito Alta (Força AGENTS.md)

6.2. Discussão Ética, Limites e Segurança: Os Quatro Pilares Críticos



1. Alucinação e Código Inseguro

Modelos de IA operam por previsão estatística e não por execução formal. O risco de *"package hallucination"* (sugestão de bibliotecas inexistentes que podem ser sequestradas via typosquatting) e padrões de segurança defasados (injeção SQL, supressão de exceções) foi contido no FixTime pela **proibição de dependências não justificadas** e testes rigorosos com compiladores estritos (Java 21 e TypeScript).



2. Vazamento de Dados e Privacidade (LGPD)

O envio de contexto para servidores em nuvem de terceiros impõe severos riscos de exposição de segredos e PII. No FixTime, adotou-se o **isolamento de segredos em variáveis de ambiente (.env)** e o uso exclusivo de **dados puramente sintéticos** nos testes e sementes (nomes e e-mails fictícios), garantindo total observância à Lei Geral de Proteção de Dados (Lei nº 13.709/2018).



3. Propriedade Intelectual (IP) e Copyright

O treinamento de LLMs em repositórios abertos cria o risco de contaminação por código licenciado sob copyleft restritivo (GPL/AGPL). O FixTime garantiu que os agentes atuassem como **rascunhadores sintáticos** da arquitetura, contratos e regras de negócio de autoria humana original da equipe, protegendo a titularidade intelectual do projeto.



4. Centralidade da Homologação Humana

O maior risco na engenharia com IA é o *"automation bias"* (a falsa confiança na correção do código pela sua aparência limpa). O fluxo SDD consolidou o princípio de que **a IA propõe, mas a equipe humana homologa**: nenhum PR foi aceito sem testes 100% verdes e revisão minuciosa entre pares.

7 Relato de Experiência e Aprendizados da Equipe

A condução do FixTime com 6 integrantes comprovou que a maior alavanca de produtividade em software colaborativo não reside na velocidade isolada de digitação de código, mas sim na **solidez da especificação técnica inicial (SDD) e na governança rígida de branches e Pull Requests**. A dissociação inicial entre o backend (Spring Boot) e o frontend (React) funcionou sem conflitos porque os contratos de entrada e saída já haviam sido selados na especificação. A cultura de 104 testes automatizados conferiu segurança psicológica para refatorações ousadas, demonstrando que ferramentas de IA são catalisadores excepcionais quando aliadas ao rigor metodológico da engenharia de software tradicional.